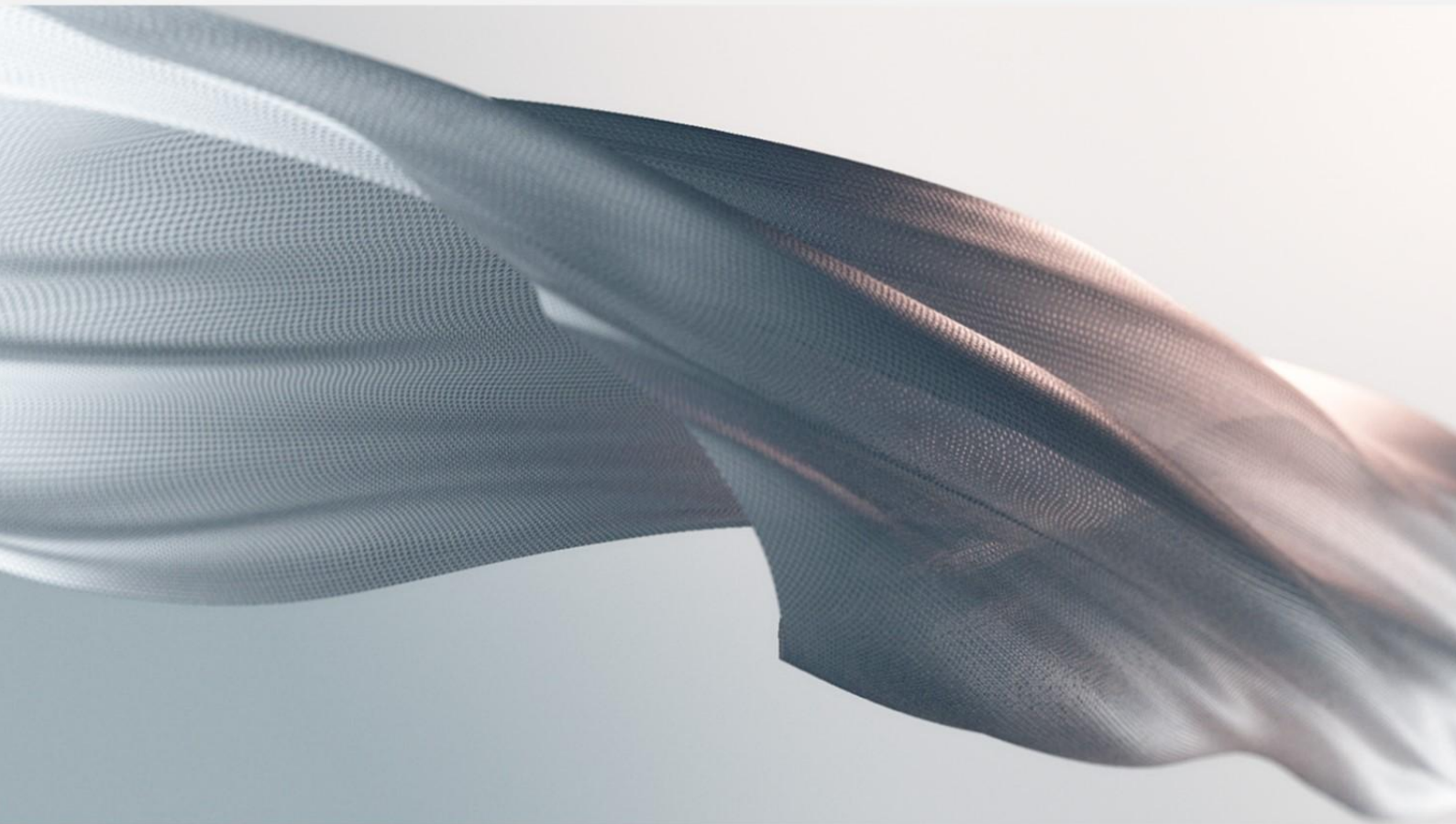

MikroDuino GPIO SDK Library

MikroDuino Studio

AVR Microcontroller Development Environment



MikroDuino GPIO Library

From Registers to Modern C++

In the previous part of this book, we learned how to control the AVR GPIO hardware by manipulating the **DDRx**, **PORTx**, and **PINx** registers directly. Although this method gives complete control over the hardware, it also requires the programmer to remember numerous register names, bit positions, and microcontroller-specific details.

Consider the following statement:

```
DDRB |= (1 << DDB5);
```

To an experienced AVR programmer, its meaning is obvious: configure PB5 as an output.

To a beginner, however, it raises several questions.

- ✓ What is DDRB?
- ✓ What does DDB5 represent?
- ✓ Why are we shifting the number 1?
- ✓ Why are we using the OR operator?
- ✓ Why not simply write `DDRB = 32`?

Even experienced programmers often have to pause for a moment before understanding such statements, especially when returning to a project after several months/ As software projects become larger, readability becomes just as important as efficiency.

This is one of the primary goals of the MikroDuino SDK.

Instead of asking the programmer to manipulate hardware registers directly, the SDK provides a modern C++ interface that expresses the programmer's intention clearly.

Instead of writing

```
DDRB |= (1 << DDB5);
```

you simply write

```
GPIO::output(PB5);
```

The second statement is almost like reading an English sentence.

Configure pin PB5 as an output.

There is no ambiguity. The programmer immediately understands the purpose of the instruction.

Zero-Cost Abstraction

One concern often raised by beginners is whether library functions are slower than direct register programming.

This is a perfectly reasonable question.

In desktop programming, calling a library function usually introduces additional overhead. Parameters must be passed, the function must be called, and control eventually returns to the caller.

In embedded systems, especially on small 8-bit microcontrollers, unnecessary overhead can significantly reduce performance.

The MikroDuino GPIO library has therefore been designed as a **zero-cost abstraction**. Its purpose is **not** to hide the hardware. Instead, it provides a cleaner language for expressing the same hardware operations.

After compiler optimization, statements such as

```
GPIO::output(PB5);
```

typically generate machine code that is virtually identical to

```
DDRB |= (1 << DDB5);
```

The programmer enjoys cleaner code without sacrificing execution speed. This philosophy is one of the cornerstones of modern embedded C++ programming.

Why Use a Library at All?

Suppose you are writing a simple LED blinking program.

Using registers directly might seem acceptable.

```
DDRB |= (1<<DDB5);  
PORTB |= (1<<PORTB5);  
PORTB &= ~(1<<PORTB5);
```

Now imagine writing firmware containing

- ✓ 20 LEDs
- ✓ 12 push buttons

- ✓ 8 relays
- ✓ SPI devices
- ✓ LCD display
- ✓ EEPROM
- ✓ Sensors

spread over several AVR families.

Soon your application becomes filled with statements such as

```
DDRB
DDRC
DDRD
PORTB
PORTC
PORTD
PINB
PINC
PIND
```

Understanding the actual application logic becomes increasingly difficult.

Using the SDK,

```
GPIO::output(LED1);
GPIO::output(RELAY1);
GPIO::inputPullup(BUTTON1);
GPIO::set(RELAY1);
GPIO::clear(RELAY2);
```

The program begins to describe **what it wants to do**, rather than **how the hardware performs it**. This is precisely the purpose of abstraction.

The Library Does Not Replace Hardware Knowledge

An important point should be emphasized here. The MikroDuino SDK does **not** attempt to hide the AVR hardware.

On the contrary.

The SDK assumes that the programmer understands the hardware registers discussed in the previous part of this chapter. Every GPIO function ultimately manipulates one or more of the familiar registers:

- ✓ DDR_x
- ✓ PORT_x
- ✓ PIN_x

Understanding these registers makes it much easier to understand what every SDK function actually does behind the scenes. The library simply performs these operations for you in a safer, cleaner, and more portable manner.

Including the GPIO Library

In our previous chapter we have shown that to use the GPIO library, which is bundled with MikroDuino IDE, under the MikroDuino folder. The library is wrapped in **namespace MikroDuino**. So you will include the **gpio.hpp** header file first.

```
#include <mikroduino/gpio.hpp>
using namespace MikroDuino
```

Understanding Pin Constants

One of the first things you will notice when programming with the MikroDuino SDK is that functions such as `GPIO::output()`, `GPIO::set()`, and `GPIO::read()` expect a **pin constant** as their parameter.

For example,

```
GPIO::output(PB5);
GPIO::set(PB5);
```

At first glance, PB5 may appear to be nothing more than another name for the number **5**. In reality, it is much more sophisticated than that.

Understanding these pin constants will help you appreciate one of the fundamental design decisions behind the MikroDuino SDK.

From Register Programming to Object-Oriented Thinking

In traditional AVR programming, every GPIO operation requires the programmer to know two separate pieces of information.

First, the **port** to which the pin belongs. Second, the **bit number** within that port.

For example, configuring PB5 as an output requires

```
DDRB |= (1 << DDB5);
```

Notice that this statement contains two separate identifiers:

- **DDRB** tells us that the pin belongs to **Port B**.
- **DDB5** tells us that we wish to modify **Bit 5**.

The programmer must mentally combine these two pieces of information every time a GPIO operation is performed.

The MikroDuino SDK simplifies this process.

Instead of asking for the port and bit separately, it asks for a single object that completely identifies the pin.

```
GPIO::output(PB5);
```

The constant PB5 already contains everything the library needs to know.

A Pin Is More Than a Number

Consider the following statement:

```
GPIO::set(5);
```

What does the number **5** represent?

Is it

- ✓ Port A, Bit 5?
- ✓ Port B, Bit 5?
- ✓ Port C, Bit 5?
- ✓ Arduino Digital Pin 5?
- ✓ ADC Channel 5?

The compiler has absolutely no way of knowing.

This ambiguity is one of the biggest problems with APIs that identify GPIO pins using ordinary integers.

Now consider

```
GPIO::set(PB5);
```

There is no ambiguity whatsoever.

The programmer immediately understands

Set Port B, Pin 5 HIGH.

Likewise,

```
GPIO::input(PD2);  
GPIO::read(PC4);  
GPIO::toggle(PA7);
```

Each statement describes the exact hardware pin being used. The code becomes almost self-documenting.

The Meaning of PB5

The naming convention follows the same terminology used throughout every AVR datasheet.

```
PB5
P = Port
B = Port B
5 = Bit number 5
```

Similarly,

```
PC3
P = Port
C = Port C
3 = Bit 3

PD0
P = Port
D = Port D
0 = Bit 0
```

This naming scheme has been used by AVR microcontrollers for decades and is instantly recognizable to embedded programmers.

Unlike Arduino's abstract pin numbering, MikroDuino deliberately preserves the actual hardware identity of every pin. This makes it much easier to cross-reference your programs with AVR datasheets and schematic diagrams.

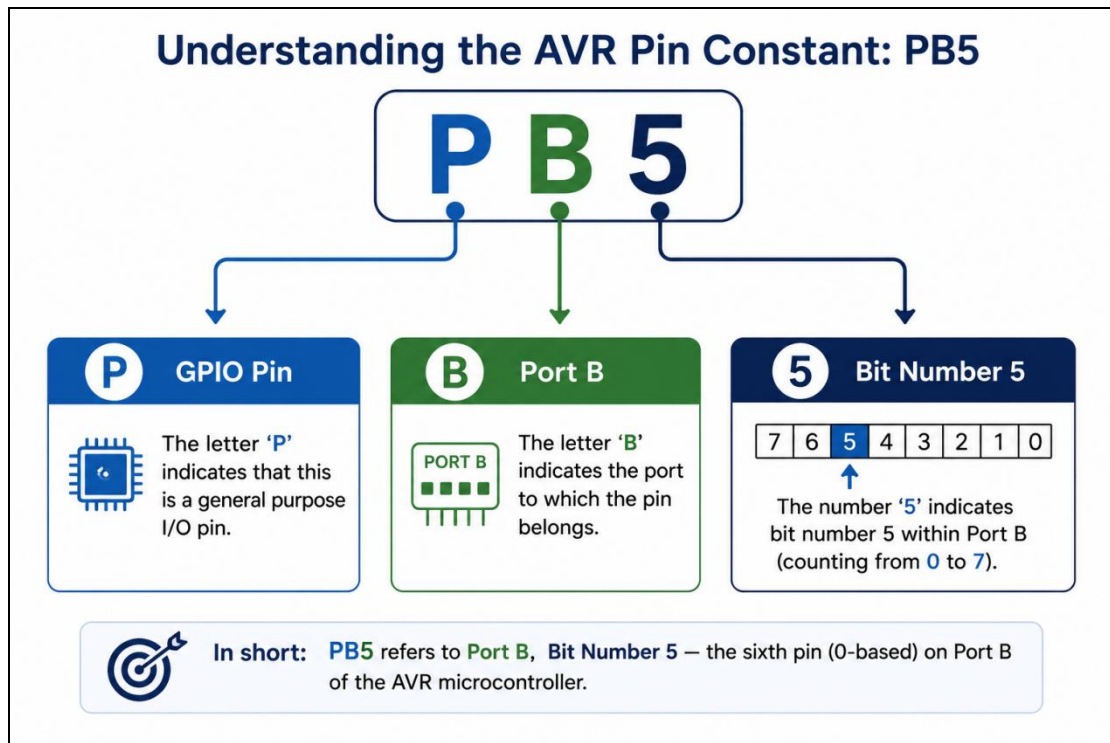
One Constant, Two Pieces of Information

Internally, the MikroDuino SDK does something rather clever. Each pin constant is represented by a single **8-bit unsigned integer** (`uint8_t`).

Within this byte, the library stores both

- ✓ the port number, and
- ✓ the bit number.

Conceptually, the value can be imagined as:



The programmer never needs to know the actual encoding. Whenever a function such as:

```
GPIO::output(PB5);
```

is called, the library automatically extracts

- ✓ which hardware port should be accessed, and
- ✓ which bit within that port should be modified.

The entire operation happens at compile time, producing extremely efficient machine code.

Why Encode Both Values?

Suppose the SDK had been designed like this:

```
GPIO::output(PORT_B, 5);
```

Every GPIO function would require two parameters.

```
GPIO::set(PORT_B, 5);
GPIO::clear(PORT_D, 2);
GPIO::read(PORT_C, 4);
```

While perfectly functional, this approach has several disadvantages.

First, every function call becomes longer. Second, the compiler must pass two parameters instead of one. This takes stack space. Third, there is always the possibility of accidentally mixing the parameters.

For example,

```
GPIO::set(PORT_D, 5);
```

instead of

```
GPIO::set(PORT_B, 5);
```

By encoding both pieces of information into a single constant, the SDK eliminates these possibilities.

Every GPIO function requires only one parameter.

```
GPIO::set(PB5);  
GPIO::clear(PD2);  
GPIO::read(PC4);
```

The result is shorter, cleaner, and easier to read, and this is how pins are named in datasheets.

Portability Across AVR Families

Another important advantage of pin constants becomes apparent when moving between different AVR devices.

Consider the three microcontrollers supported by MikroDuino.

ATmega328P

Available ports:

```
Port B  
Port C  
Port D
```

ATmega32

Available ports:

```
Port A  
Port B  
Port C  
Port D
```

ATmega128

Available ports:

```
Port A
Port B
Port C
Port D
Port E
Port F
Port G
```

The SDK automatically defines only those pin constants that exist on the selected microcontroller.

For example, PA0 does not exist on an ATmega328P because that device has no Port A. Attempting to use it results in a compile-time error rather than a subtle runtime bug. This compile-time checking helps prevent many common mistakes when moving projects between different AVR families.

Comparison with Arduino Pin Numbers

Many beginners are familiar with Arduino's numbering scheme.

```
Digital Pin 13
Digital Pin 12
Digital Pin 11
```

Although convenient for hobby projects, these numbers have no obvious relationship with the actual AVR hardware.

For example,

```
Arduino Pin 13
↓
ATmega328P
↓
PB5
```

A programmer reading

```
digitalWrite(13, HIGH);
```

must either remember or look up that Arduino pin 13 corresponds to PB5 on the ATmega328P.

With the MikroDuino SDK, there is no translation step.

```
GPIO::set(PB5);
```

The software directly reflects the hardware design.

This close relationship between source code and hardware makes debugging easier, simplifies reading datasheets, and encourages a deeper understanding of the AVR architecture.

Using Pin Constants Throughout the SDK

One of the strengths of the GPIO library is its consistency. The same pin constant is used by every GPIO function.

```
GPIO::output(PB5);  
GPIO::set(PB5);  
GPIO::clear(PB5);  
GPIO::toggle(PB5);  
GPIO::read(PB5);  
GPIO::write(PB5, true);
```

Once you know the name of a pin, you can use it throughout your entire program without worrying about register names or bit positions.

A Foundation for Higher-Level Libraries

The use of encoded pin constants extends beyond the GPIO library. Because every pin is represented by a single value, higher-level drivers—such as those for character LCDs, keypads, sensors, or communication interfaces—can accept the same pin constants as configuration parameters.

For example, a future LCD library might be initialized like this:

```
LCD lcd(  
    RS  = PB0,  
    EN  = PB1,  
    D4  = PD4,  
    D5  = PD5,  
    D6  = PD6,  
    D7  = PD7  
);
```

The LCD driver does not need to know anything about `DDRB`, `PORTD`, or individual register addresses. It simply receives pin constants, and the underlying GPIO layer performs the necessary register operations.

This consistent representation of pins across the entire SDK is one of the key design principles of MikroDuino. By treating a pin as a first-class object rather than just a number, the SDK provides an interface that is intuitive to read, efficient to compile, and portable across multiple AVR microcontroller families.

GPIO Class Design

One of the first things that may surprise programmers coming from traditional object-oriented programming is that the `GPIO` class in the MikroDuino SDK is **never instantiated**. Unlike many C++ classes, you do not create a `GPIO` object before using it. Instead, every function is called directly on the class itself.

For example, instead of writing

```
GPIO gpio;  
gpio.output(PB5);  
gpio.set(PB5);
```

you simply write

```
GPIO::output(PB5);  
GPIO::set(PB5);
```

This design is intentional and reflects the nature of embedded hardware.

Why Not Create GPIO Objects?

To answer this question, let us first think about what a GPIO pin actually represents. Unlike a software object such as a document, a network connection, or a database record, a GPIO pin is **not something that exists because we create an object**.

It already exists as a physical piece of hardware inside the microcontroller. Whether your program creates a GPIO object or not, the microcontroller will always have

- ✓ Port A
- ✓ Port B
- ✓ Port C
- ✓ Port D

and their associated hardware registers.

The library therefore does not need an object to represent them. Instead, it simply provides convenient functions that manipulate the existing hardware.

GPIO Is a Utility Class

The GPIO class is what C++ programmers often call a **utility class** or a **static utility class**. A utility class groups together a collection of closely related functions without requiring any internal state. Think of the GPIO class as a toolbox.

You do not need to build a new toolbox every time you want to tighten a screw. The toolbox already exists. You simply pick the required tool and use it.

Similarly,

```
GPIO::output(PB5);  
GPIO::set(PB5);  
GPIO::toggle(PB5);
```

are simply different "tools" grouped under the GPIO class.

What Does **static** Mean?

In C++, the keyword `static` has several meanings depending on where it is used. Inside a class, a **static member function** belongs to the class itself rather than to any particular object.

Consider the following simplified example.

```
class GPIO
{
public:

    static void output(uint8_t pin);
    static void set(uint8_t pin);
    static void clear(uint8_t pin);
};
```

Notice the keyword **static** before every function.

Because these functions are static, they are called using the scope resolution operator (::).

```
GPIO::output(PB5);
GPIO::set(PB5);
```

instead of

```
GPIO gpio;
gpio.output(PB5);
```

There is no object because none is required.

No RAM Is Consumed

One of the greatest advantages of a static utility class is that **it occupies virtually no RAM**.

Suppose we had designed the library like this.

```
GPIO led;
```

Every object occupies memory. Even if the object stored only a few bytes of information, creating dozens of GPIO objects would unnecessarily consume precious SRAM.

Remember that many AVR microcontrollers have only

- 1 KB
- 2 KB
- 4 KB

of RAM. Every byte matters.

By making the GPIO class completely static, no object storage is required. Only the program code itself occupies Flash memory. This makes the library extremely efficient.

No Constructor Required

Object-oriented programming normally begins by constructing an object.

For example,

```
Serial serial;  
LCD lcd;  
Motor motor;
```

Each object performs some initialization. GPIO, however, has nothing to initialize. The hardware already exists. The registers already exist. The library merely accesses them. For this reason, the GPIO class intentionally prevents object creation. Attempting to write

```
GPIO gpio;
```

results in a compile-time error because the *constructor has been deleted*. This reminds the programmer that GPIO is not meant to be instantiated and should always be used through its static interface.

Think of GPIO as a Control Panel

A useful analogy is to imagine the dashboard of an industrial machine.

The dashboard contains

- ✓ Start button
- ✓ Stop button
- ✓ Emergency switch
- ✓ Indicator lamps

You never "create" the dashboard.

It already exists. You simply press the required button whenever you need it. The GPIO class behaves in exactly the same manner. It provides a collection of controls that operate the microcontroller's existing hardware.

Better Code Readability

Another advantage of this design is improved readability. Consider the following statement.

```
GPIO::toggle(PB5);
```

The programmer immediately understands

Toggle GPIO pin PB5.

Now compare this with

```
gpio.toggle(PB5);
```

The reader now has to ask another question.

Which GPIO object is this?

Is it

- ✓ LED GPIO?
- ✓ LCD GPIO?
- ✓ SPI GPIO?
- ✓ Another GPIO driver?

Since there is only one GPIO peripheral inside the microcontroller, using the class name directly makes the code clearer.

One Interface for Every AVR

Another design objective of the MikroDuino SDK is portability.

Whether your project targets an

- ✓ ATmega16
- ✓ ATmega32
- ✓ ATmega328P
- ✓ ATmega64
- ✓ ATmega128

the programmer always writes the same code.

```
GPIO::output(PB5);  
GPIO::set(PB5);  
GPIO::read(PD2);
```

The library automatically selects the correct hardware registers based on the encoded pin constant and the target microcontroller. This means your application code remains unchanged even when moving between supported AVR devices, while the SDK handles the device-specific register mapping behind the scenes.

When Should You Use Direct Register Programming?

Although the MikroDuino SDK should be your preferred interface for everyday application development, there are situations where direct register manipulation is still appropriate.

For example:

- ✓ Studying how AVR hardware works at the register level.
- ✓ Reading examples directly from AVR datasheets or application notes.
- ✓ Configuring specialized peripheral registers that are not yet wrapped by the SDK.
- ✓ Writing highly optimized, timing-critical routines where every instruction cycle has been carefully analyzed.

In these situations, the knowledge gained in the previous sections of this chapter becomes invaluable. For nearly all general-purpose GPIO operations, however, the SDK provides a cleaner, safer, and more portable solution. It expresses your intent clearly, reduces the likelihood of programming errors, and produces code that is easier to understand, maintain, and extend—without sacrificing the efficiency expected in embedded systems programming.

Configuring Pin Direction

Before a GPIO pin can be used, the microcontroller must know whether the pin will act as an **input** or an **output**.

As discussed in Part 2, this decision is controlled by the **Data Direction Register (DDR_x)**. A bit value of **1** configures the corresponding pin as an output, while a value of **0** configures it as an input.

The MikroDuino SDK provides three functions for configuring the direction of a GPIO pin:

```
GPIO::output(pin);  
GPIO::input(pin);  
GPIO::inputPullup(pin);
```

These three functions cover almost every digital input/output application encountered in embedded systems.

GPIO::output()

Purpose: Configures the specified GPIO pin as a digital output.

Once configured as an output, the pin can source or sink current to external circuitry and may be controlled using `GPIO::set()`, `GPIO::clear()`, `GPIO::toggle()`, or `GPIO::write()`.

Syntax

```
GPIO::output(uint8_t pin);
```

Parameter

Parameter	Description
pin	Any valid MikroDuino pin constant such as PB5, PD2, PC0, etc.

Return Value: This function does not return any value.

Hardware Equivalent

Calling

```
GPIO::output(PB5);
```

is equivalent to

```
DDRB |= (1 << DDB5);
```

The function simply sets the corresponding bit in the appropriate DDR register.

How It Works

Internally the library performs four simple steps.

- ✓ Decode the pin constant.
- ✓ Determine the port.
- ✓ Determine the bit position.
- ✓ Set the corresponding DDR bit.

Example 1 — Configure a Single LED

```
#include <mikroduino/gpio.hpp>
using namespace MikroDuino;

int main()
{
    GPIO::output(PB5);
    while(true)
    {
        GPIO::toggle(PB5);
    }
}
```

Example 2 — Configure Multiple LEDs

```
GPIO::output(PB0);  
GPIO::output(PB1);  
GPIO::output(PB2);  
GPIO::output(PB3);
```

Notice how readable this is.

Compare with

```
DDRB |= (1<<DDB0);  
DDRB |= (1<<DDB1);  
DDRB |= (1<<DDB2);  
DDRB |= (1<<DDB3);
```

Example 3 — Different Ports

```
GPIO::output(PB5);  
GPIO::output(PC2);  
GPIO::output(PD7);
```

The programmer never needs to remember which DDR register belongs to which port.

The SDK handles everything automatically.

Typical Applications

Configure

- ✓ LEDs
- ✓ Buzzers
- ✓ Relays
- ✓ Motors
- ✓ Chip Select lines
- ✓ LCD control pins
- ✓ SPI outputs

Common Mistakes

Forgetting to configure the pin first

Incorrect

```
GPIO::set(PB5);
```

The pin is still an input, the default mode.

Correct

```
GPIO::output(PB5);  
GPIO::set(PB5);
```

Confusing output() with set()

Many beginners believe

```
GPIO::output(PB5);
```

will drive the pin HIGH.

It does not. It merely changes the direction. The actual voltage is controlled separately.

```
GPIO::set(PB5);
```

or

```
GPIO::clear(PB5);
```

Performance

The compiler typically converts this function into the same machine instructions that would have been produced by direct register manipulation.

There is essentially no runtime overhead.

Comparison

Raw AVR	MikroDuino SDK
<code>DDRB</code>	<code>= (1<<DDB5);</code>

The SDK version is considerably easier to understand while generating equally efficient code.

GPIO::input()

Purpose

Configures a GPIO pin as a **floating digital input**.

The output driver is disconnected, allowing external circuitry to control the voltage on the pin.

Syntax

```
GPIO::input(uint8_t pin);
```

Hardware Equivalent

```
GPIO::input(PD2);
```

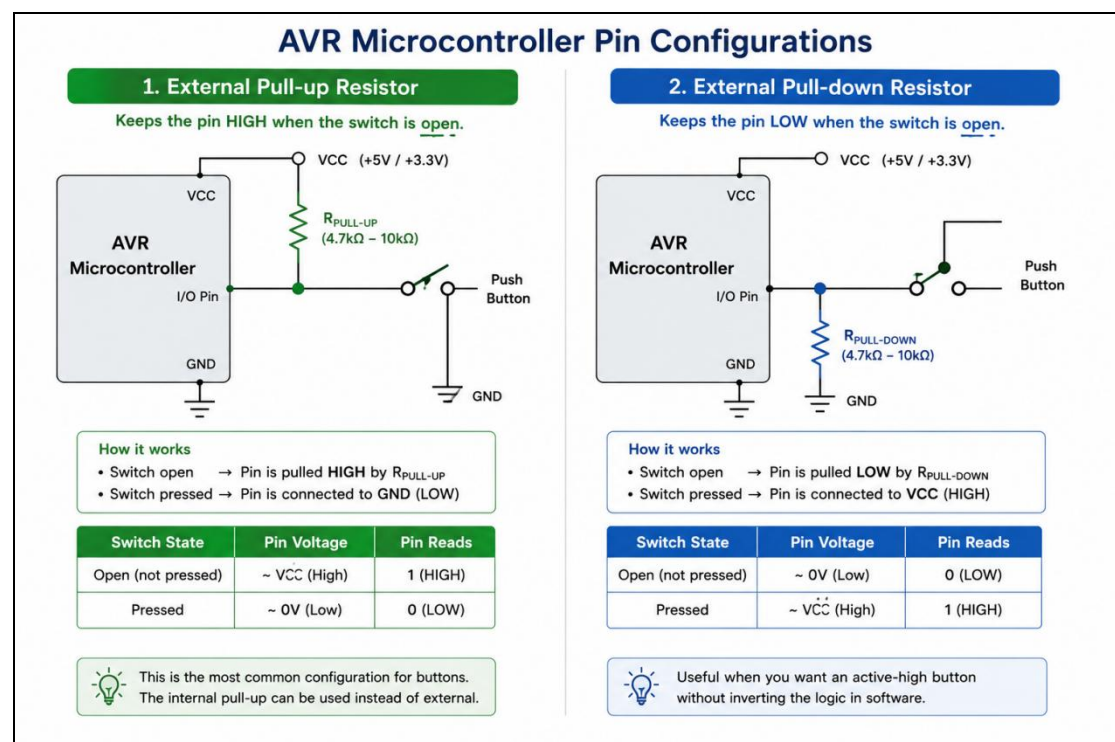
corresponds to

```
DDRD &= ~(1 << DDD2);  
PORTD &= ~(1 << PORTD2);
```

The DDR bit is cleared to select input mode, and the PORT bit is also cleared so that the internal pull-up resistor remains disabled.

What Is a Floating Input?

A floating input is **not connected internally** to either VCC or GND. Its voltage depends entirely on the external circuit. If nothing is connected, the voltage may randomly fluctuate due to electrical noise. For this reason, floating inputs should generally be avoided unless the external hardware already provides a pull-up or pull-down resistor.



Example

External switch with resistor.

```
GPIO::input(PD2);

while(true)
{
    if(GPIO::read(PD2))
    {
        GPIO::set(PB5);
    }
}
```

Typical Applications

- ✓ External logic circuits
- ✓ TTL outputs
- ✓ Sensors with built-in pull-ups
- ✓ Open-collector interfaces with external resistors

Common Mistake

Using

```
GPIO::input()
```

for a push button connected directly to ground. Without an external resistor, the input will float when the button is released, producing unpredictable readings.

GPIO::inputPullup()

This is probably the **most frequently used input configuration** in AVR programming.

Purpose

Configures a GPIO pin as an input while simultaneously enabling the AVR's built-in pull-up resistor. This eliminates the need for an external resistor in many applications, particularly push-button circuits.

Hardware Equivalent

```
GPIO::inputPullup(PD2);
```

becomes

```

DDRD  &= ~(1<<DDD2) ;
PORTD |= (1<<PORTD2) ;

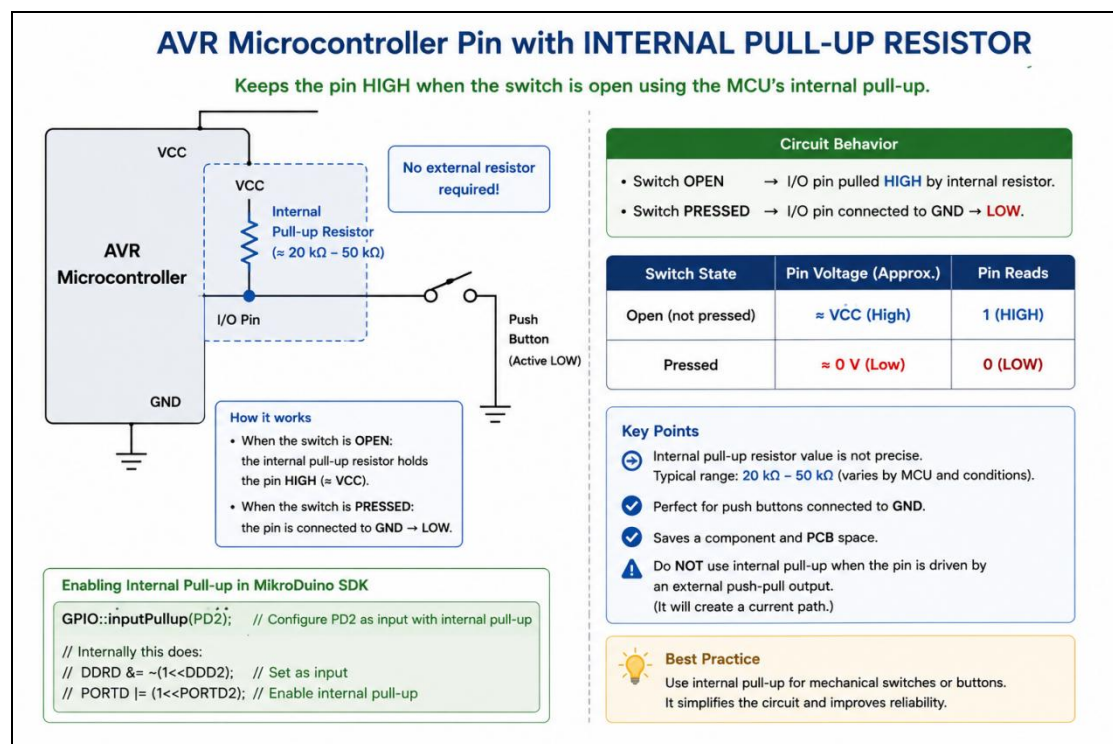
```

The DDR bit selects input mode, while the PORT bit enables the internal pull-up resistor.

Why Use Pull-ups?

When a switch is open, an input pin without a defined voltage can randomly read HIGH or LOW due to electrical noise. The internal pull-up resistor solves this problem by gently pulling the input toward VCC whenever the switch is not pressed.

When the button is pressed, it connects the pin directly to ground, overriding the weak pull-up and producing a clean LOW signal.



Example

```

GPIO::output(PB5);

GPIO::inputPullup(PD2);

while(true)
{
    GPIO::write(PB5, !GPIO::read(PD2));
}

```

Notice the ! operator.

Because the button is wired to ground, pressing it produces a LOW. The logical NOT reverses the value so that the LED turns on when the button is pressed.

Typical Applications

Nearly every push-button interface in AVR systems.

- ✓ User buttons
- ✓ Keypads
- ✓ DIP switches
- ✓ Limit switches
- ✓ Mechanical contacts

Comparison of Input Modes

Function	Internal Pull-up	Typical Use
<code>GPIO::input()</code>	No	External circuitry already defines the signal level
<code>GPIO::inputPullup()</code>	Yes	Push buttons, switches, open-drain outputs

Controlling Digital Outputs

In the previous section, we learned how to configure a GPIO pin as an output. However, simply configuring the direction of a pin does not produce any visible effect.

A pin configured as an output is much like a water tap connected to a pipe. Installing the tap does not cause water to flow; someone must still turn the handle to let the water flow or stop it.

Similarly, after a GPIO pin has been configured as an output, the application must decide what electrical signal should appear on that pin.

- ✓ Should it output a HIGH voltage?
- ✓ Should it output a LOW voltage?
- ✓ Should it change from HIGH to LOW?
- ✓ Should it simply invert its present state?

These four operations form the basis of almost every digital output application, whether you are blinking an LED, driving a relay, controlling a buzzer, enabling a motor driver, or selecting an SPI device.

The MikroDuino SDK provides four simple functions for these operations:

```
GPIO::set(pin);  
GPIO::clear(pin);
```



```
GPIO::write(pin, value);
GPIO::toggle(pin);
```

Together, these functions replace nearly all direct manipulation of the **PORTx** and **PINx** registers.

GPIO::set()

Purpose

Drives the selected output pin to the **HIGH** logic level. For a typical AVR operating at 5 V, this places approximately **5 V** on the output pin. When operating at 3.3 V, the output voltage is approximately **3.3 V**.

Syntax

```
GPIO::set(uint8_t pin);
```

Parameters

Parameter	Description
pin	GPIO pin constant such as PB5, PD2, or PC0.

Return Value

None.

Hardware Equivalent

```
GPIO::set(PB5);
```

is equivalent to

```
PORTB |= (1 << PORTB5);
```

Internally, the SDK locates the correct PORT register from the encoded pin constant and sets the appropriate bit.

What Actually Happens?

If the pin has already been configured as an output,

```
DDRB5 = 1
```

then

```
GPIO::set(PB5);
```

changes

```
PORTB5 = 1
```

which causes the output driver inside the AVR to connect the pin to **VCC**.

Example 1 – Turn an LED ON

```
GPIO::output(PB5);  
GPIO::set(PB5);
```

Example 2 – Activate a Relay

```
GPIO::output(PD6);  
GPIO::set(PD6);
```

Example 3 – Enable a Motor Driver

```
GPIO::output(PB1);  
GPIO::set(PB1);
```

Common Mistake

Many beginners write

```
GPIO::set(PB5);
```

without first configuring the pin as an output.

Always remember

```
GPIO::output(PB5);  
GPIO::set(PB5);
```

Performance

This function compiles into the same register operation you would normally write by hand.

GPIO::clear()

Purpose

Drives the selected output pin to the **LOW** logic level.

Syntax

```
GPIO::clear(pin);
```

Hardware Equivalent

```
GPIO::clear(PB5);
```

becomes

```
PORTB &= ~(1 << PORTB5);
```

Internally, the library clears only the required PORT bit while leaving every other pin on the same port unchanged.

Example 1 – Turn LED OFF

```
GPIO::clear(PB5);
```

Example 2 – De-energize Relay

```
GPIO::clear(PD6);
```

GPIO::write()

Among all output functions, **write()** is probably the one used most frequently in real-world applications.

Purpose

Writes either HIGH or LOW to an output pin depending upon a Boolean value.

Syntax

```
GPIO::write(pin, value);
```

Parameters

Parameter	Description
pin	GPIO pin
value	true = HIGH, false = LOW

Why Does This Function Exist?

Suppose you have

```
bool ledState;
```

Without `write()`, you would need

```
if(ledState)
    GPIO::set(PB5);
else
    GPIO::clear(PB5);
```

Every time.

Instead,

```
GPIO::write(PB5, ledState);
```

The code immediately communicates your intent.

Example – Mirror a Push Button

```
GPIO::output(PB5);

GPIO::inputPullup(PD2);

while(true)
{
    GPIO::write(PB5, !GPIO::read(PD2));
}
```

One line replaces an entire `if-else` statement.

Example – Multiple Outputs

```
GPIO::write(RELAY1, relayEnabled);
```

```
GPIO::write(BUZZER, alarmActive);  
GPIO::write(LED, systemReady);
```

Notice how naturally the code reads.

Hardware Equivalent

Internally, the function simply chooses between

```
GPIO::set(pin);
```

and

```
GPIO::clear(pin);
```

based on the Boolean parameter.

Best Practice

Whenever the output value already exists as a variable, prefer `GPIO::write()` over an explicit `if-else` statement. It produces cleaner, shorter, and easier-to-maintain code.

GPIO::toggle()

Purpose

Reverses the present state of an output pin. HIGH becomes LOW. LOW becomes HIGH.

Syntax

```
GPIO::toggle(pin);
```

Hardware Equivalent

Many programmers expect this to perform

```
PORTB ^= (1<<PORTB5);
```

However, the MikroDuino SDK takes advantage of a unique AVR hardware feature.

Internally it performs the equivalent of

```
PINB = (1 << PINB5);
```

Writing a **1** to the corresponding bit in the `PINx` register instructs the AVR hardware to toggle the output automatically. This is a single-instruction, atomic operation that is both faster and safer than performing a read-modify-write sequence in software.

Example 1 – Blink an LED

```
GPIO::output(PB5);

while(true)
{
    GPIO::toggle(PB5);

    delay_ms(500);
}
```

Example 2 – Heartbeat Indicator

```
ISR(TIMER0_OVF_vect)
{
    GPIO::toggle(PB0);
}
```

Every timer overflow automatically inverts the LED state.

Typical Applications

- ✓ Status LEDs
- ✓ Heartbeat indicators
- ✓ Square-wave generation
- ✓ Debug output pins
- ✓ Timing measurements with an oscilloscope

Comparison of Output Functions

Function	Effect	Typical Application
<code>GPIO::set(pin)</code>	Drive pin HIGH	Turn on LED, energize relay, assert chip select
<code>GPIO::clear(pin)</code>	Drive pin LOW	Turn off LED, release relay, deassert chip select
<code>GPIO::write(pin, value)</code>	Drive HIGH or LOW according to a Boolean value	Mirror program variables to outputs, sensor-driven control

Function	Effect	Typical Application
<code>GPIO::toggle(pin)</code>	Invert the current output state	Blinking LEDs, heartbeat indicators, square waves

Choosing the Right Function

Although all four functions ultimately manipulate the same hardware registers, each one expresses a different programming intention.

- Use `GPIO::set()` when you know the output must become HIGH.
- Use `GPIO::clear()` when the output must become LOW.
- Use `GPIO::write()` when the desired state is already stored in a variable or expression.
- Use `GPIO::toggle()` when you simply want to invert the current state without caring whether it is presently HIGH or LOW.

By selecting the function that best matches your intent, your code becomes more expressive, easier to review, and simpler to maintain. One of the key design goals of the MikroDuino SDK is to let the source code describe **what** your application is trying to accomplish, while the library efficiently handles the underlying register manipulations.

AVR Bit Manipulation Macros

Throughout this chapter, we have used the standard C++ bitwise operators to manipulate individual bits in AVR hardware registers. Although these operators form the foundation of all embedded programming, the AVR software ecosystem also provides a number of helper macros that make register programming shorter, clearer, and less error-prone.

If you study examples from AVR datasheets, Atmel application notes, or open-source libraries, you will frequently encounter expressions such as

```
_BV(PB5)

bit_is_set(PIND, PIND2)

loop_until_bit_is_clear(UCSRA, RXC)
```

At first sight these may appear to be special compiler instructions. In reality, they are simply **macros** provided by the AVR-LibC library.

Macros are expanded by the C preprocessor before the compiler begins compiling the program. They introduce no runtime overhead and usually generate exactly the same machine code as the equivalent hand-written expressions.

Although the MikroDuino SDK eliminates the need for most direct register manipulation, understanding these macros is valuable because they are widely used throughout the AVR ecosystem.

What Is a Macro?

A macro is simply a textual substitution.

For example,

```
#define LED PB5
```

means that every occurrence of

```
LED
```

is replaced by

```
PB5
```

before compilation begins.

Macros may also accept parameters.

For example,

```
#define SQUARE(x) ((x) * (x))
```

allows

```
SQUARE(5)
```

to become

```
((5) * (5))
```

before the compiler ever sees the program.

The AVR-LibC bit manipulation helpers work in exactly the same way.

The **`__BV()`** Macro

The most famous AVR macro is undoubtedly

```
__BV(bit)
```

The name stands for **Bit Value**.

It is defined approximately as


```
#define _BV(bit) (1 << (bit))
```

Therefore,

```
_BV(PB5)
```

is exactly equivalent to

```
(1 << PB5)
```

Both produce

```
00100000
```

when PB5 equals 5.

Why Does `_BV()` Exist?

Some programmers believe

```
_BV(PB5)
```

is easier to read because it immediately communicates

“Generate the bit value for PB5.”

Compare

```
PORTB |= (1 << PB5);
```

with

```
PORTB |= _BV(PB5);
```

Both compile into identical machine code. The choice is largely one of programming style.

Example

Configure PB5 as output.

```
DDRB |= _BV(DDB5);
```

Turn LED on.

```
PORTB |= _BV(PORTB5);
```

Turn LED off.

```
PORTB &= ~_BV(PORTB5);
```

bit_is_set()

Checking whether a bit is set is one of the most common operations in embedded software.

Without the macro we write

```
if (PIND & (1 << PIND2))
{
    ...
}
```

AVR-LibC provides

```
bit_is_set(PIND, PIND2)
```

which performs exactly the same operation.

Example

```
if (bit_is_set(PIND, PIND2))
{
    LED_ON();
}
```

Many programmers find this version considerably easier to read.

It almost resembles an English sentence.

If bit is set...

bit_is_clear()

The opposite operation is equally common.

Instead of

```
if (!(PIND & (1 << PIND2)))
```

we may write

```
if (bit_is_clear(PIND, PIND2))
{
```

```
...  
}
```

This is especially useful for push buttons connected using pull-up resistors.

Example

```
GPIO::inputPullup(PD2);  
  
if(bit_is_clear(PIND, PIND2))  
{  
    GPIO::set(PB5);  
}
```

The expression immediately communicates

Execute if the button is pressed.

Waiting Macros

Sometimes a program must simply wait until a hardware flag changes.

AVR-LibC provides two useful macros.

```
loop_until_bit_is_set(register, bit);
```

and

```
loop_until_bit_is_clear(register, bit);
```

Example

Wait until UART transmission completes.

```
loop_until_bit_is_set(UCSRA, TXC);
```

Equivalent C++ code

```
while(!(UCSRA & (1<<TXC))  
{  
}
```

The macro is shorter and often easier to understand.

Frequently Used AVR Register Macros

Although not officially part of AVR-LibC, many embedded projects define their own helper macros.

Examples include

```
#define SETBIT(REG,BIT)      ((REG) |= (1<<(BIT)))  
  
#define CLEARBIT(REG,BIT)   ((REG) &= ~(1<<(BIT)))  
  
#define TOGGLEBIT(REG,BIT)  ((REG) ^= (1<<(BIT)))  
  
#define CHECKBIT(REG,BIT)   ((REG) & (1<<(BIT)))
```

These macros merely wrap the familiar bitwise operations.

Example

Instead of

```
PORTB |= (1<<PB5);
```

one may write

```
SETBIT(PORTB, PB5);
```

Likewise,

```
CLEARBIT(PORTB, PB5);  
  
TOGGLEBIT(PORTB, PB5);
```

Although these macros can improve readability, they are **not standardized**. Different projects often use different names such as `SET_BIT`, `BIT_SET`, `sbi`, or `BITSET`, so you should not assume they will be available in every codebase.

Port Mask Macros

Larger projects sometimes manipulate several bits simultaneously.

Typical helper macros include

```
#define SETMASK(REG,MASK)    ((REG) |= (MASK))  
  
#define CLEARMASK(REG,MASK) ((REG) &= ~(MASK))
```

Example

```
SETMASK(PORTB, 0b00101100);
```

or

```
CLEARMASK(PORTD, 0b00001111);
```

These are especially useful when configuring multiple GPIO pins or peripheral control bits in a single operation.

Should You Use These Macros?

For direct register programming, these macros can make code shorter and often more expressive. They are widely used in AVR examples and can improve readability when working close to the hardware.

However, for **general GPIO programming**, the MikroDuino SDK remains the recommended approach. Compare the following:

Task	Raw Register	AVR Macro	MikroDuino SDK
Configure PB5 as output	<code>`DDRB</code>	<code>= (1 << DDB5);`</code>	<code>`DDRB</code>
Set PB5 HIGH	<code>`PORTB</code>	<code>= (1 << PORTB5);`</code>	<code>SETBIT(PORTB, PORTB5);</code>
Clear PB5	<code>PORTB &= ~(1 << PORTB5);</code>	<code>CLEARBIT(PORTB, PORTB5);</code>	<code>GPIO::clear(PB5);</code>
Toggle PB5	<code>PINB = (1 << PINB5);</code>	<i>(no standard macro)</i>	<code>GPIO::toggle(PB5);</code>
Read PD2	<code>PIND & (1 << PIND2)</code>	<code>bit_is_set(PIND, PIND2)</code>	<code>GPIO::read(PD2)</code>

The MikroDuino SDK provides a **higher level of abstraction** while still generating efficient machine code. Rather than requiring the programmer to think in terms of registers and bit masks, it allows the code to express the intended operation directly.

For this reason, throughout the remainder of this book, we will generally use the MikroDuino SDK for GPIO operations, while still introducing AVR-LibC macros where they are commonly encountered in datasheets, third-party libraries, or low-level peripheral programming. This balanced approach gives you the best of both worlds: a solid understanding of the underlying hardware and a clean, maintainable coding style for real-world embedded applications.

Project 1 — Digital Input and Output Control

LED and Push Button

Objective

In this project, we will build one of the most fundamental embedded applications—a push button controlling an LED.

Although the circuit is extremely simple, it demonstrates almost every important GPIO operation learned in this chapter.

You will learn how to:

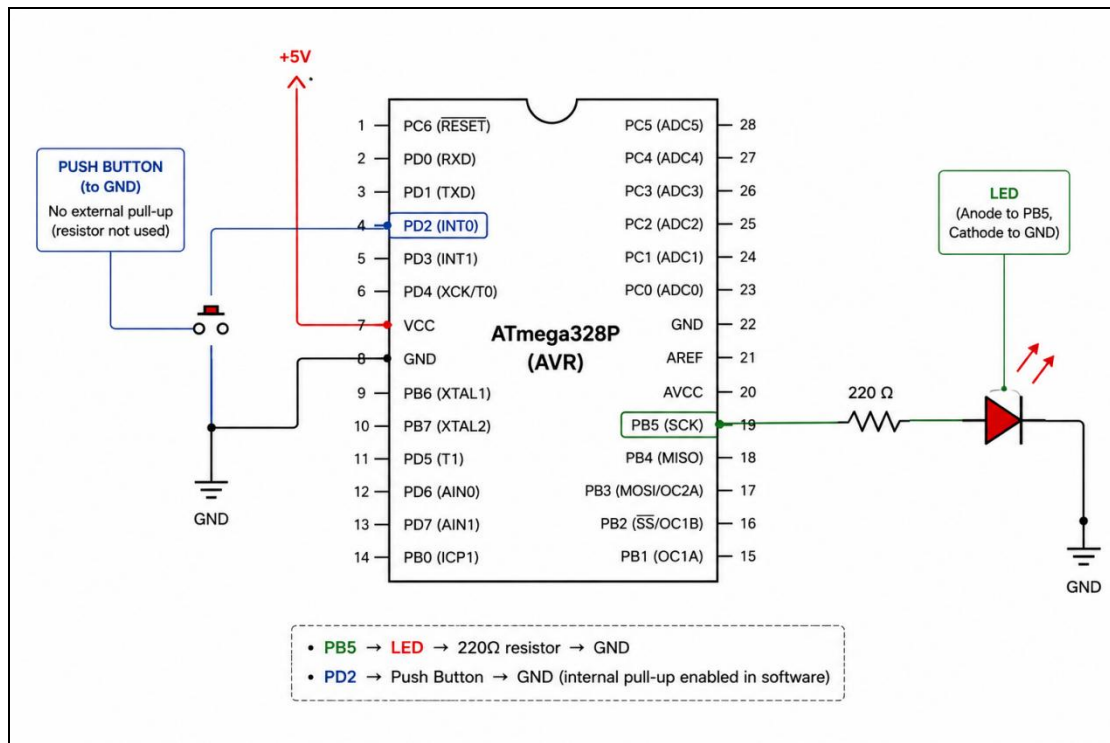
- Configure GPIO pins as inputs and outputs.
- Use the AVR's internal pull-up resistor.
- Read the state of a push button.
- Control an LED.
- Use `GPIO::write()` to simplify application logic.
- Appreciate how the SDK eliminates direct register manipulation.

This example forms the basis of countless real-world embedded systems, including industrial control panels, medical equipment, home automation systems, and robotics.

Hardware Required

Quantity	Component
1	AVR development board (ATmega328P/ATmega32/etc.)
1	LED
1	220 Ω resistor
1	Push button
—	Connecting wires

Circuit Diagram



Notice that **no external resistor** is connected to the push button because we shall use the AVR's internal pull-up resistor.

Program

```
#include <mikroduino/mikroduino.hpp>

using namespace MikroDuino;

int main()
{
    // Configure hardware
    GPIO::output(PB5);           // LED
    GPIO::inputPullup(PD2);      // Push button

    while (true)
    {
        // Button is active LOW
        GPIO::write(PB5, !GPIO::read(PD2));
    }
}
```

Program Walkthrough

Step 1 — Include the SDK

```
#include <mikroduino/mikroduino.hpp>
```

This header includes all the commonly used MikroDuino libraries, including the GPIO library.

Step 2 — Configure the LED

```
GPIO::output(PB5);
```

PB5 is configured as an output.

Internally, this performs the equivalent of

```
DDRB |= (1 << DDB5);
```

Step 3 — Configure the Button

```
GPIO::inputPullup(PD2);
```

This configures PD2 as an input while enabling the AVR's internal pull-up resistor.

Equivalent AVR code

```
DDRD &= ~(1 << DDD2);  
PORTD |= (1 << PORTD2);
```

Step 4 — Read the Button

```
GPIO::read(PD2)
```

returns

- true when the pin is HIGH
- false when the pin is LOW

Because our switch connects the pin to ground,

Button	Pin State	GPIO::read()
Released	HIGH	true
Pressed	LOW	false

Step 5 — Invert the Logic

```
!GPIO::read(PD2)
```


The logical NOT (!) reverses the result.

Button	GPIO::read()	!GPIO::read()
Released	true	false
Pressed	false	true

Now the Boolean value directly matches the desired LED state.

Step 6 — Drive the LED

```
GPIO::write(PB5, !GPIO::read(PD2));
```

This single statement

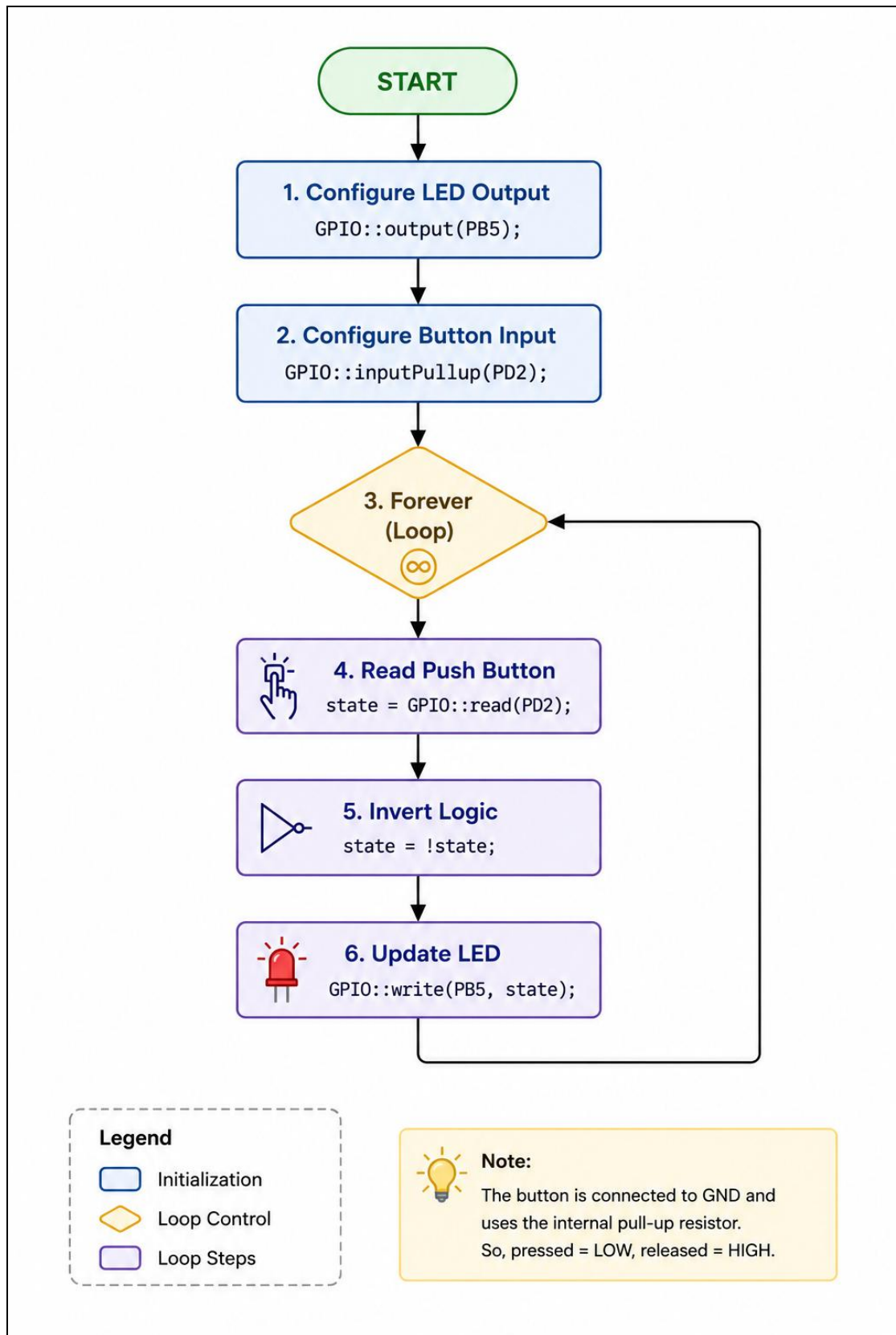
- reads the button,
- inverts the logic,
- writes the result to the LED.

Without `GPIO::write()`, the same code would be

```
if (!GPIO::read(PD2))
    GPIO::set(PB5);
else
    GPIO::clear(PB5);
```

The SDK version is considerably shorter and more expressive.

Program Flow



What You Learned

This simple project demonstrates the use of

- ✓ GPIO::output()
- ✓ GPIO::inputPullup()
- ✓ GPIO::read()
- ✓ GPIO::write()

all working together.

Project 2 — Knight Rider LED Scanner

This second project demonstrates GPIO output functions in a more dynamic application.

Objective

Create the famous "Knight Rider" or "Larson Scanner" LED effect using eight LEDs connected to Port B.

The project demonstrates

- ✓ configuring multiple outputs,
- ✓ repeatedly using GPIO::set() and GPIO::clear(),
- ✓ arrays,
- ✓ loops,
- ✓ timing,
- ✓ reusable functions.

Hardware

Eight LEDs connected to

```
PB0  
PB1  
PB2  
PB3  
PB4  
PB5  
PB6  
PB7
```

Each LED should have its own 220 Ω current-limiting resistor.

Program

```
#include <mikroduino/mikroduino.hpp>  
  
using namespace MikroDuino;
```

```

constexpr uint8_t leds[] =
{
    PB0,
    PB1,
    PB2,
    PB3,
    PB4,
    PB5,
    PB6,
    PB7
};

void showLED(uint8_t index)
{
    // Turn all LEDs OFF
    for (uint8_t i = 0; i < 8; i++)
        GPIO::clear(leds[i]);

    // Turn selected LED ON
    GPIO::set(leds[index]);
}

int main()
{
    // Configure all LEDs
    for (uint8_t i = 0; i < 8; i++)
        GPIO::output(leds[i]);

    while (true)
    {
        // Left to right
        for (uint8_t i = 0; i < 8; i++)
        {
            showLED(i);
            delay_ms(100);
        }

        // Right to left
        for (uint8_t i = 6; i >= 1; i--)
        {
            showLED(i);
            delay_ms(100);
        }
    }
}

```

Concepts Demonstrated

This project brings together many ideas introduced throughout the chapter:

- ✓ Using an array of GPIO pin constants.
- ✓ Configuring multiple pins with a loop.
- ✓ Creating reusable functions (showLED()).
- ✓ Driving outputs with GPIO::set() and GPIO::clear().
- ✓ Using loops to generate visual effects.
- ✓ Organizing code into small, readable building blocks.

It also illustrates one of the major strengths of the MikroDuino SDK: the application logic remains focused on **what** the program should do—display a moving light—rather than **how** each individual AVR register must be manipulated.

These two projects make an excellent conclusion to the GPIO chapter. The first introduces both digital inputs and outputs in the simplest possible way, while the second demonstrates how the GPIO library scales naturally to larger applications without sacrificing readability. Together, they reinforce the idea that once the underlying hardware is understood, the MikroDuino SDK provides a clean, expressive, and efficient way to build real-world embedded systems.